

Rapport de Conception Technique

Système de Gestion de l'Internat AIAC

(ResiAIAC)

Table des matières

1	Étude bibliographique	3
1.1	Introduction	3
1.2	Technologies et Outils Backend	3
1.2.1	Spring Boot	3
1.2.2	PostgreSQL	3
1.2.3	Redis	3
1.2.4	SeaweedFS	4
1.2.5	Keycloak	4
1.2.6	Spring Security & RBAC	4
1.2.7	MapStruct	4
1.2.8	JUnit 5, Mockito & MockMvc	5
1.3	Technologies et Outils Frontend	5
1.3.1	Angular	5
1.3.2	RxJS	5
1.3.3	TailwindCSS	5
1.4	Infrastructure, Déploiement et Concepts	6
1.4.1	Docker & Docker Compose	6
1.4.2	Kubernetes (K8s)	6
1.4.3	GitHub Actions	6
1.4.4	Systèmes Distribués & API REST	6
1.4.5	Gestion d'État (State Management)	6
1.4.6	Mise en Cache (Caching)	6
1.5	Conclusion et Références Bibliographiques	6
2	Étude des opportunités et Analyse de l'existant	8
2.1	Situation Actuelle et Problématique	8
2.2	Synthèse des Problèmes et Opportunités	8
2.3	Analyse Comparative des Outils Existants	8
3	Conception Statistique et Modèle de Données	9
3.1	Diagramme des Classes – Spécifications du Modèle de Données	9
3.2	Modèle Physique de Données (Schéma de Base de Données)	11

4	Architecture Applicative Backend	13
4.1	Organisation en Couches	13
4.2	Séparation DTO / Entité	13
4.3	Gestion des Identifiants Composites	13
4.4	Gestion des Erreurs	14
5	Architecture de Déploiement	14
6	Modélisation des Processus (Diagrammes de Séquences)	15
6.1	Module A : Admission et Validation des Dossiers	15
6.1.1	Scénario A1 : Dépôt de Documents par l'Étudiant	15
6.1.2	Scénario A2 : Traitement et Validation de Dossier par le Responsable	16
6.2	Module B : Réservation et Attribution des Chambres	17
6.2.1	Scénario B1 : Réservation de Chambre par l'Étudiant	17
6.3	Module C : Réclamations et Demandes de Changement	18
6.3.1	Scénario C1 : Dépôt Direct d'une Réclamation par l'Étudiant	18
6.3.2	Scénario C2 : Création de Réclamation depuis l'Espace Étudiant	18
6.3.3	Scénario C3 : Suivi de l'Avancement par l'Étudiant	19
6.3.4	Scénario C4 : Traitement et Clôture par le Manager de Résidence	20
6.4	Module D : Gestion des Départs et États des Lieux	21
6.4.1	Scénario D : Processus Administratif de Libération de Chambre	21
6.5	Module E : Administration, RBAC et Audit	22
6.5.1	Scénario E1 : Gestion Applicative des Profils Utilisateurs	22
6.5.2	Scénario E2 : Gestion Fine des Rôles (RBAC)	23
6.5.3	Scénario E3 : Consultation des Journaux d'Audit Sécurisés	23

1 Étude bibliographique

1.1 Introduction

Cette analyse bibliographique présente un panorama complet des technologies, frameworks, outils et concepts modernes de développement logiciel mis en œuvre dans le cadre du projet **ResiAIAC**. Ce système intègre des composants backend et frontend avancés, des architectures distribuées hautement scalables et des pratiques d'intégration et de déploiement continus (DevOps) de standard industriel.

1.2 Technologies et Outils Backend

1.2.1 Spring Boot

Définition : Spring Boot est un framework Java d'entreprise open-source, conçu sur l'écosystème Spring, qui élimine la complexité de configuration des applications autonomes prêtes pour la production. Il repose sur le paradigme de *"convention plutôt que configuration"* (Convention over Configuration).

Caractéristiques clés :

- **Auto-configuration :** Résolution et instanciation automatiques des beans requis en fonction des dépendances présentes.
- **Serveur Embarqué :** Intégration par défaut de serveurs d'applications (Tomcat, Jetty ou Undertow) éliminant le besoin de serveurs d'applications externes.
- **Gestion des Dépendances :** Utilisation des "starters" Maven/Gradle standardisant les architectures de dépendance.

Intérêt pratique : Permet le développement ultra-rapide de microservices extensibles et d'API REST résilientes et sécurisées.

1.2.2 PostgreSQL

Définition : Système de gestion de base de données relationnelle et objet (SGBDR) open-source de classe entreprise. Il est mondialement reconnu pour son respect rigoureux des standards SQL, sa fiabilité exceptionnelle et sa richesse fonctionnelle.

Caractéristiques clés :

- **Transactions ACID :** Support complet et robuste garantissant l'intégrité absolue des données.
- **Modèles de Données Hybrides :** Gestion native et optimisée du format JSONB permettant d'associer la flexibilité du NoSQL à la rigueur du relationnel.
- **Extensibilité :** Support d'extensions avancées (PostGIS pour les données spatiales, pgvector pour l'IA).

1.2.3 Redis

Définition : Redis (Remote Dictionary Server) est une base de données en mémoire, clé-valeur, ultra-rapide. Elle est principalement exploitée comme cache applicatif, courtier de messages (Message Broker) ou stockage de sessions.

Caractéristiques clés :

- Structures de données hautement optimisées (listes, hashes, sets triés).
- Latence inférieure à la milliseconde grâce au stockage en RAM.
- Persistance configurable (RDB/AOF) pour prévenir la perte accidentelle de données.

1.2.4 SeaweedFS

Définition : Système de stockage distribué d'objets, hautement scalable et efficace pour le stockage de milliards de petits fichiers, inspiré de l'architecture Haystack de Facebook.

Caractéristiques clés :

- Séparation stricte de la gestion des métadonnées (Master Server) et du stockage physique des fichiers (Volume Servers).
- Compatibilité totale avec l'API Amazon S3.
- Réplication native et basculement automatique garantissant une haute disponibilité.

1.2.5 Keycloak

Définition : Solution open-source de gestion des identités et des accès (IAM), assurant l'authentification et l'autorisation via les protocoles standards OAuth2 et OpenID Connect (OIDC).

Caractéristiques clés :

- **Émission de jetons JWT :** Génère des jetons signés contenant les rôles et informations de l'utilisateur authentifié, vérifiés côté backend sans appel systématique au serveur d'identité.
- **Gestion des *clients* et des rôles :** Permet de définir des applications clientes distinctes (backend, outils de test) et une hiérarchie de rôles applicatifs.
- **Déploiement autonome :** Hébergé au sein du cluster Kubernetes aux côtés des services applicatifs, avec sa propre base de données dédiée.

Intérêt pratique : Centralise l'authentification pour un déploiement mono-tenant (une seule école), sans besoin de fédération SSO entre plusieurs fournisseurs d'identité.

1.2.6 Spring Security & RBAC

Définition : Module de l'écosystème Spring assurant l'authentification et le contrôle d'accès aux ressources d'une application. Couplé à Keycloak, il agit en *Resource Server* validant les jetons JWT entrants.

Caractéristiques clés :

- **Contrôle d'accès basé sur les rôles (RBAC) :** Restriction fine des opérations selon le rôle de l'utilisateur (Étudiant, Responsable, Manager, Administrateur).
- **Journalisation d'audit :** Traçabilité des actions sensibles effectuées sur le système, consultable par les administrateurs.
- **Approvisionnement des comptes (Provisioning) :** Création conjointe du compte Keycloak et de l'entité *Utilisateur* au moment de l'admission, afin d'éviter la circulation de données personnelles non chiffrées dans le jeton JWT.

1.2.7 MapStruct

Définition : Générateur de code Java, exécuté à la compilation, produisant des *mappers* pour convertir automatiquement les entités JPA vers leurs Objets de Transfert de Données (DTO) et inversement.

Caractéristiques clés :

- Génération de code à la compilation (pas de réflexion à l'exécution), garantissant des performances proches d'un mapping manuel.
- Réduction du code répétitif (*boilerplate*) associé à la conversion entité/DTO sur chaque endpoint REST.

1.2.8 JUnit 5, Mockito & MockMvc

Définition : Ensemble de bibliothèques de test formant la pile de vérification automatisée du backend : JUnit 5 comme moteur d'exécution, Mockito pour l'isolation des dépendances, AssertJ pour des assertions lisibles, et MockMvc pour tester la couche web sans démarrer de serveur réel.

Caractéristiques clés :

- **Tests de service (unitaires) :** Chaque `ServiceImpl` est testé isolément, dépendances (*repository*, *mapper*) simulées via `@Mock`; les appels statiques de `ResourceFetcher` sont simulés avec `mockStatic` (Mockito 5 / `mockito-inline`).
- **Tests de contrôleur (tranche web) :** Chaque contrôleur est testé via `@WebMvcTest` et `MockMvc`, le service associé étant remplacé par `@MockitoBean`; seuls le routage, la (dé)sérialisation JSON et les codes de statut sont vérifiés ici, la logique métier étant déjà couverte par les tests de service.
- **Filtres de sécurité désactivés à ce niveau :** Les tests de contrôleur démarrent avec `addFilters = false`, l'authentification Keycloak/JWT étant hors périmètre de cette tranche de tests.
- **Couverture des champs énumérés :** Vérification explicite du comportement des champs de type *enum* (ex. `EtatChambre`, `EtatDocument`), y compris leur acceptation actuelle de la valeur `NULL` — point identifié comme perfectible.

Intérêt pratique : Exécution automatisée à chaque contribution via GitHub Actions, avant intégration au code principal.

1.3 Technologies et Outils Frontend

1.3.1 Angular

Définition : Framework applicatif structuré et typé, développé par Google, utilisant TypeScript pour concevoir des applications web monopages (SPA) robustes, performantes et scalables au niveau entreprise.

Caractéristiques clés :

- Architecture modulaire stricte basée sur les composants réutilisables.
- Injection de dépendances native pour une modularité accrue et une testabilité maximale.
- *Two-Way Data Binding* unifiant de manière transparente le modèle et la vue.

1.3.2 RxJS

Définition : Bibliothèque réactive mettant en œuvre le pattern Observer et les flux d'observables pour orchestrer de manière asynchrone les événements et les requêtes de données.

Caractéristiques clés :

- Gestion fluide des requêtes HTTP concurrentes et du temps réel (WebSockets).
- Large palette d'opérateurs de transformation et de filtrage (`switchMap`, `debounceTime`, `combineLatest`).

1.3.3 TailwindCSS

Définition : Framework CSS orienté "utilitaires d'abord" (Utility-First), permettant de concevoir des interfaces graphiques modernes et réactives directement dans le balisage HTML.

Caractéristiques clés :

- Évite la prolifération de feuilles de style complexes et redondantes.

- Optimisation draconienne de la taille des bundles via l'élimination automatique des classes inutilisées (PurgeCSS).

1.4 Infrastructure, Déploiement et Concepts

1.4.1 Docker & Docker Compose

Définition : Docker standardise l'emballage des applications et de leur environnement d'exécution complet dans des conteneurs isolés, garantissant le principe *"build once, run anywhere"*. Docker Compose permet la définition multi-conteneur via un unique fichier déclaratif (`compose.yml`).

1.4.2 Kubernetes (K8s)

Définition : Orchestrateur de conteneurs open-source standard de l'industrie, conçu pour automatiser le déploiement, la mise à l'échelle automatique (Auto-scaling) et la gestion des applications conteneurisées.

1.4.3 GitHub Actions

Définition : Plateforme d'automatisation CI/CD intégrée à GitHub pour exécuter des pipelines de tests automatisés, de validation de code (Linter) et de déploiement automatique sur chaque événement Git.

1.4.4 Systèmes Distribués & API REST

Définition : Architecture distribuée structurant les services indépendamment, communiquant via le protocole HTTP sans état (Stateless) en s'appuyant sur les verbes HTTP standards.

1.4.5 Gestion d'État (State Management)

Définition : Centralisation et prévisibilité des données applicatives (par exemple via NgRx) pour assurer la cohérence des vues sur les interfaces utilisateur complexes.

1.4.6 Mise en Cache (Caching)

Définition : Stratégie d'optimisation visant à éliminer les goulots d'étranglement de la base de données en conservant temporairement en RAM les données hautement sollicitées.

1.5 Conclusion et Références Bibliographiques

Le choix de cette pile technologique moderne garantit la création d'une application performante, modulaire et hautement disponible.

- **Spring Boot** : <https://spring.io/projects/spring-boot>
- **PostgreSQL** : <https://www.postgresql.org/docs/>
- **Redis** : <https://redis.io/docs/>
- **SeaweedFS** : <https://github.com/seaweedfs/seaweedfs>
- **Keycloak** : <https://www.keycloak.org/documentation>
- **Spring Security** : <https://docs.spring.io/spring-security/reference/>
- **MapStruct** : <https://mapstruct.org/documentation/>
- **Docker** : <https://docs.docker.com/>

- **Kubernetes** : <https://kubernetes.io/docs/>
- **GitHub Actions** : <https://docs.github.com/en/actions>
- **Angular** : <https://angular.dev/>

2 Étude des opportunités et Analyse de l'existant

2.1 Situation Actuelle et Problématique

La gestion de la résidence universitaire de l'AIAC repose sur un écosystème hétérogène d'outils cloisonnés. L'application de gestion administrative actuelle est fermée aux étudiants, forçant ces derniers à passer par des processus manuels (cahier de réclamations physique) ou des systèmes déconnectés (formulaires BDE pour la réservation des chambres).

2.2 Synthèse des Problèmes et Opportunités

La fragmentation actuelle engendre plusieurs dysfonctionnements majeurs :

1. **Perte de temps et d'efficacité** : La double saisie des données papier vers Excel augmente la charge administrative.
2. **Erreurs humaines** : La transcription manuelle induit des incohérences de données.
3. **Absence de traçabilité** : Complexité de suivi de l'historique et de l'état des réclamations.
4. **Isolement informationnel** : Manque total de visibilité unifiée sur l'occupation de l'internat.

2.3 Analyse Comparative des Outils Existants

Le tableau ci-dessous dresse l'état des lieux des solutions de contournement actuellement utilisées par les acteurs :

Outil Actuel	Acteurs	Objectif du Processus	Limites Identifiées
Papier / Cahier	Étudiant, Responsable	Saisie des dossiers, réclamations	Perte physique, dégradation
Excel	Manager, Responsable	Suivi des réclamations	Erreurs de formules, pas de partage
Téléphone	Logisticiens, Équipes	Prise de décision urgente	Absence d'historique écrit
Email	Collaborateurs	Envoi de pièces justificatives	Risque de non-réponse, classement
WhatsApp	Manager, Personnel	Échanges informels de terrain	Non professionnel, non structuré

TABLE 1 – Cartographie des outils actuels et leurs limites

3 Conception Statistique et Modèle de Données

3.1 Diagramme des Classes – Spécifications du Modèle de Données

Pour remplacer l'application existante et centraliser l'information, le modèle de données de **Re-siAIAC** est structuré autour des entités et relations clés suivantes :

Utilisateur : Représente toute personne accédant au système (Étudiant, Manager, Responsable, Administrateur). Contient l'historique complet de ses réservations, de ses documents administratifs et de ses réclamations.

Document : Pièce justificative déposée par l'étudiant pour son dossier d'admission (carte d'identité, attestation, etc.). Il possède un état de validation (*Aucun*, *En attente*, *Validé*, *Invalide*) et une note sur sa validité.

Chambre : Entité physique caractérisée par sa capacité, son état d'occupation et sa localisation (Étage et Bâtiment). Elle est rattachée aux réservations et réclamations.

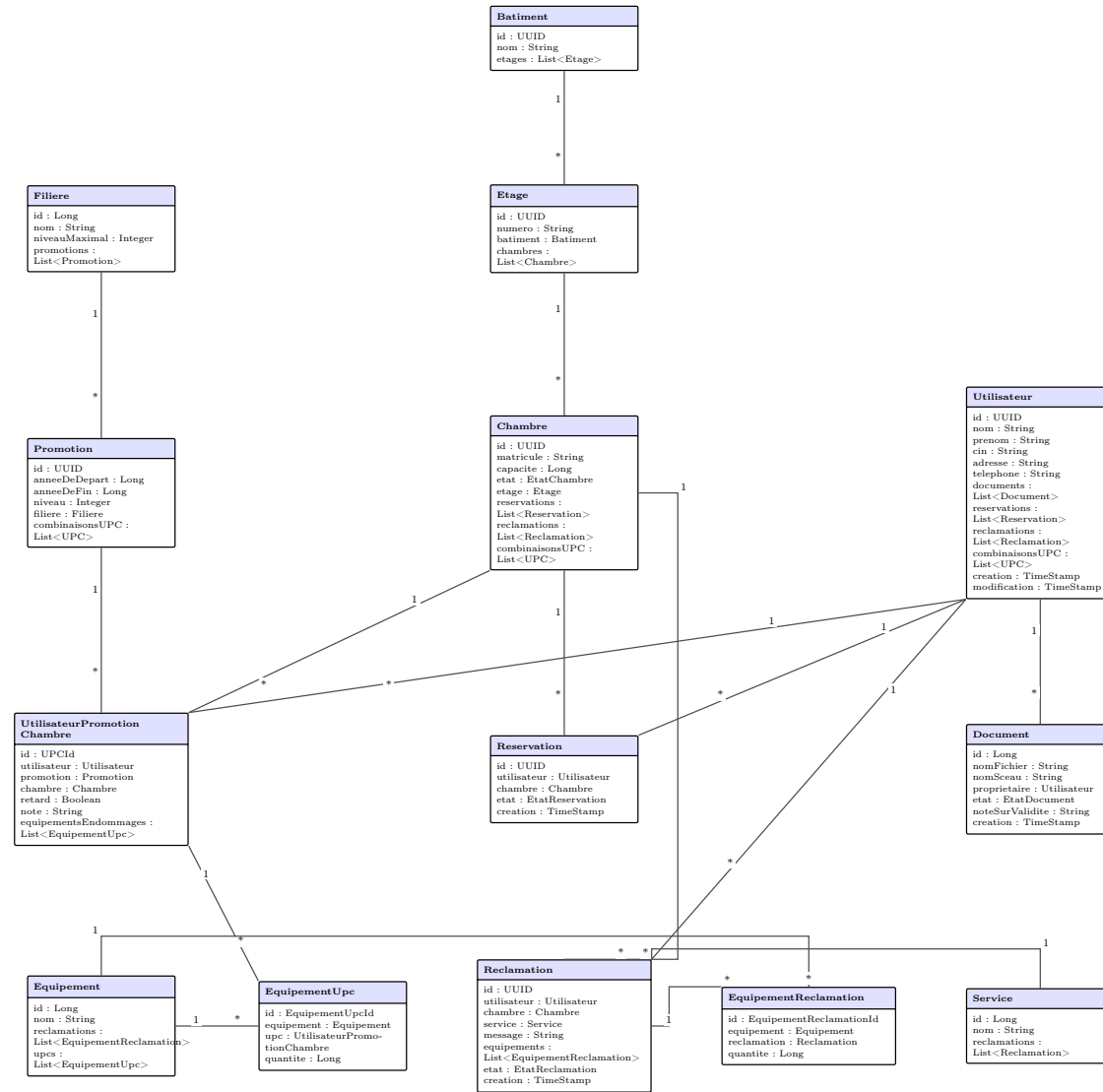
Reservation : Trace historique de l'affectation d'une chambre à un étudiant pour une période donnée.

Reclamation : Ticket ouvert par un étudiant concernant un problème technique dans sa chambre (lié à une entité **Service** et à des équipements défectueux).

UtilisateurPromotionChambre (UPC) : Table d'association majeure modélisant la répartition des étudiants par promotion et par chambre pour assurer un suivi historique rigoureux.

Le diagramme de classes UML ci-après (Figure 3.1) formalise l'ensemble de ces entités, leurs attributs ainsi que les multiplicités des relations qui les lient.

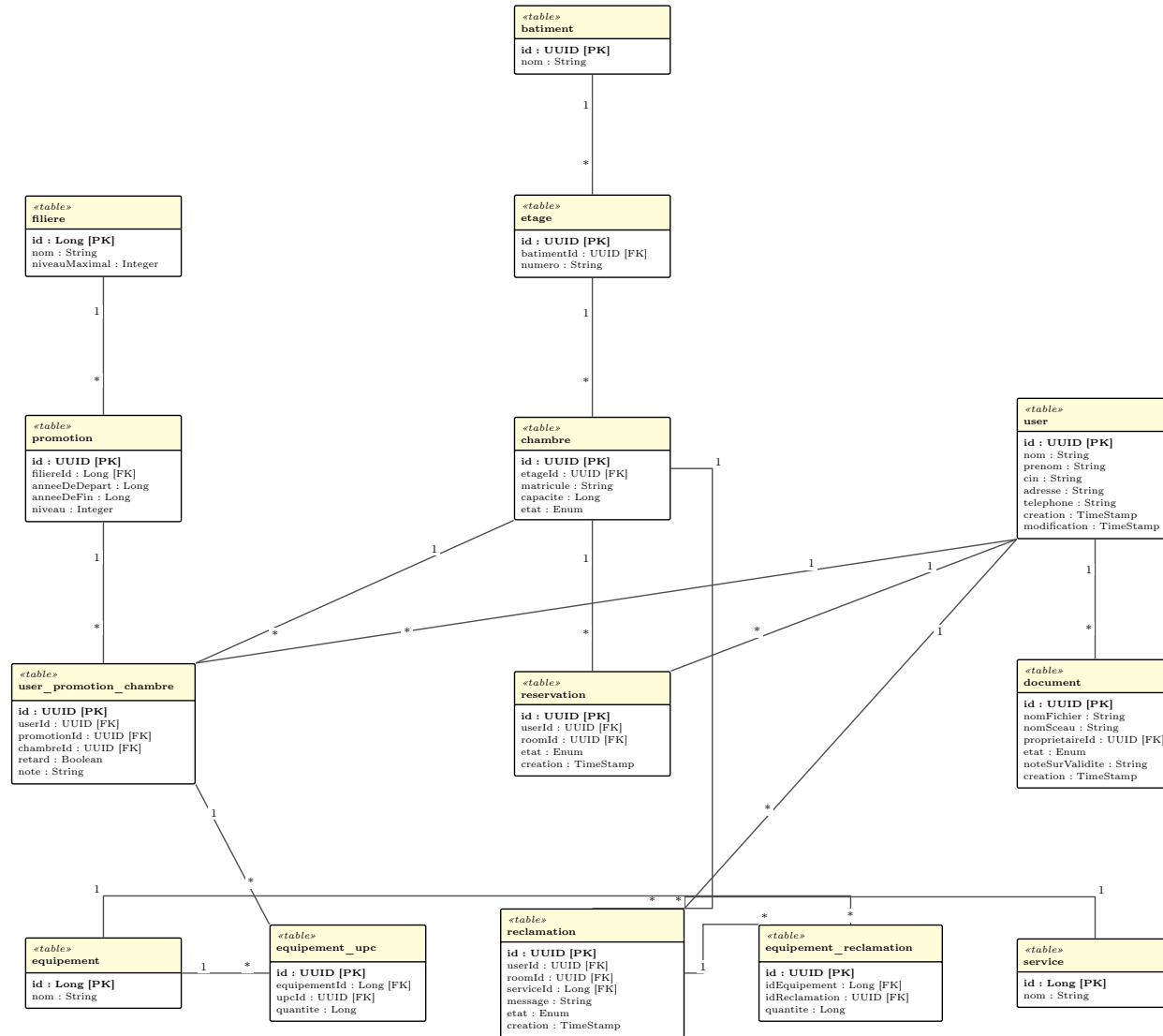
Figure 1 – Diagramme de Classes UML (Modèle Logique)



3.2 Modèle Physique de Données (Schéma de Base de Données)

Le diagramme suivant (Figure 3.2) présente la transposition du modèle logique en un schéma physique de base de données, sous forme de diagramme de classes UML où chaque classe porte le stéréotype « table ». Les attributs identifiants sont marqués **[PK]** (clé primaire) et les références vers d'autres tables **[FK]** (clé étrangère), conformément aux conventions de modélisation physique en UML.

Figure 2 – Diagramme de Classes UML (Modèle Physique / Schéma de Base de Données)

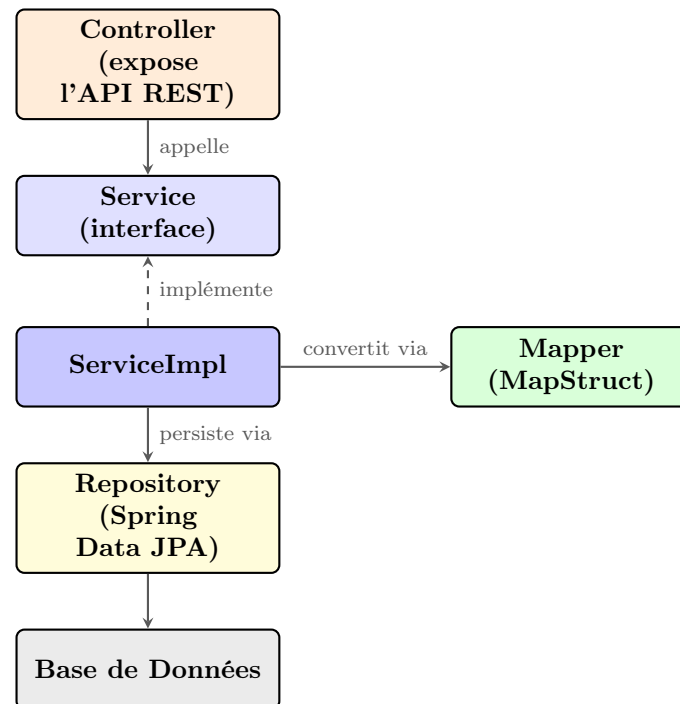


4 Architecture Applicative Backend

4.1 Organisation en Couches

Chaque module métier du backend (une entité et ses opérations associées) suit rigoureusement la même organisation en couches, illustrée par la Figure 4.1. Le contrôleur REST ne dépend que de l'**interface** du service, jamais de son implémentation concrète : ce découplage permet de faire évoluer ou remplacer la logique métier d'un service sans modifier son point d'entrée, et facilite l'écriture de tests isolés à chaque couche (voir §1.2.8).

Figure 4 – Organisation en Couches d'un Module Backend



4.2 Séparation DTO / Entité

Les entités JPA ne transitent jamais directement par l'API REST. Chaque module expose deux objets de transfert distincts, convertis par un *mapper* MapStruct dédié (voir §1.2.7) :

- **XxxDto** : représentation de sortie, retournée par les endpoints de lecture.
- **XxxUpdateRequest** : représentation d'entrée, utilisée pour la création et la mise à jour, validée avant conversion vers l'entité.

Cette séparation évite d'exposer la structure interne des entités (relations JPA, champs techniques) et permet de faire évoluer le modèle de données sans casser le contrat d'API.

4.3 Gestion des Identifiants Composites

Trois entités du modèle de données reposent sur une clé primaire composite, portée par une classe d'identifiant dédiée : `UtilisateurPromotionChambreId`, `EquipementUpId` et `EquipementReclamationId`. Par convention, les méthodes `getById` et `delete` des services concernés acceptent les composants bruts de la clé (identifiants simples) et reconstituent la clé composite **au sein de la couche service**, plutôt que de laisser le contrôleur assembler cet objet technique. La méthode `update`, à l'inverse, accepte directement l'identifiant composite déjà construit ; cette asymétrie, actuellement assumée dans l'implémentation, est identifiée comme un point à harmoniser.

4.4 Gestion des Erreurs

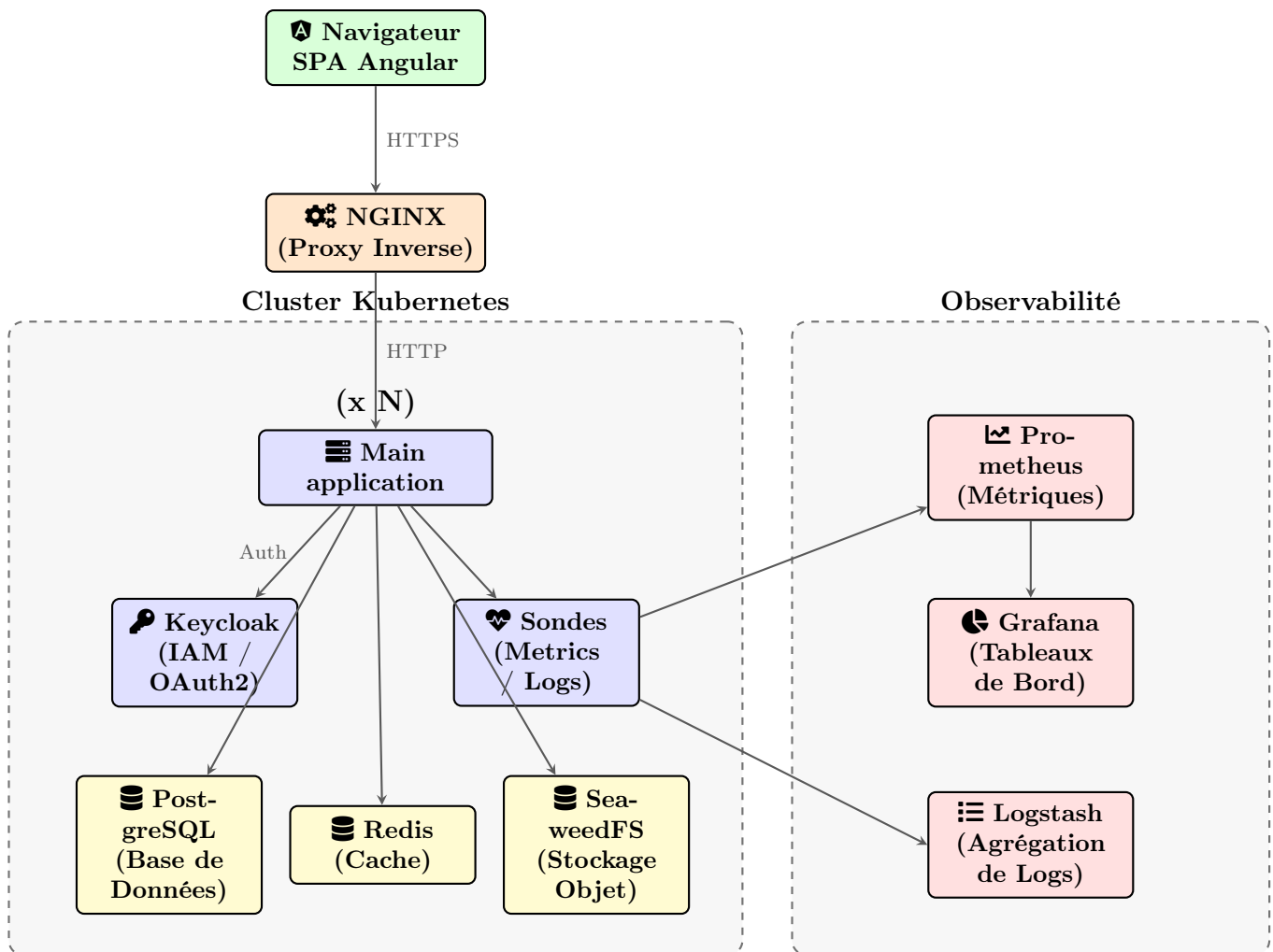
La récupération d'une ressource par son identifiant passe systématiquement par l'utilitaire générique `ResourceFetcher.fetchResource(id, repository, resourceType)`, qui centralise la logique "récupérer ou lever une exception" et évite sa duplication dans chaque service. Lorsque la ressource est introuvable, une `ResourceNotFoundException` (exception applicative dédiée) est levée.

À ce stade, aucun gestionnaire d'exceptions global (`@ControllerAdvice` / `@ExceptionHandler`) n'est encore en place : les réponses d'erreur de l'API ne sont donc pas encore uniformisées (format de corps de réponse, code HTTP associé à chaque type d'exception). Cette centralisation est prévue lors de la phase de sécurisation de l'API, aux côtés de la validation des entrées et du RBAC.

5 Architecture de Déploiement

Le diagramme de déploiement suivant (Figure 5) illustre l'architecture technique cible de **ResiAIAC**, organisée en couches : couche client (SPA Angular), couche de routage/entrée (NGINX), couche applicative conteneurisée et orchestrée par Kubernetes (services Spring Boot et Keycloak pour l'IAM), couche de persistance et de stockage (PostgreSQL, Redis, SeaweedFS), couche d'observabilité (Prometheus, Grafana, la pile Logstash) et enfin la chaîne d'intégration et de déploiement continu (CI/CD).

Figure 5 – Diagramme de Déploiement de l'Infrastructure ResiAIAC



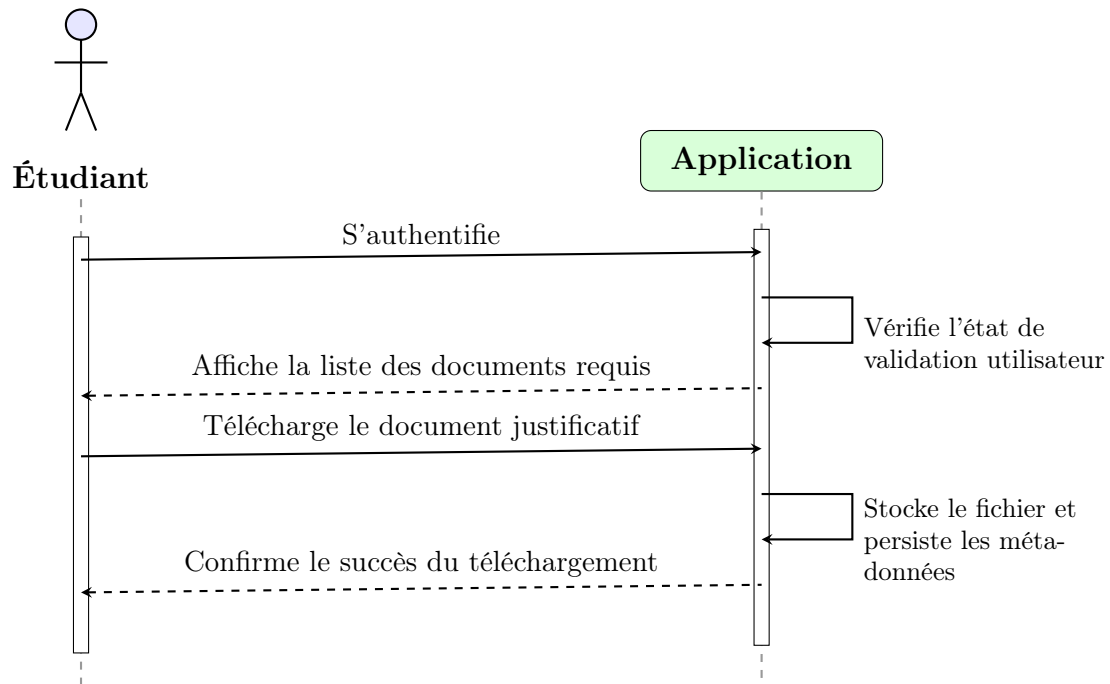
6 Modélisation des Processus (Diagrammes de Séquences)

Afin de garantir une cohérence graphique et conceptuelle parfaite, l'ensemble des scénarios a été homogénéisé selon le modèle standardisé et direct : **Acteur** → **Application**.

6.1 Module A : Admission et Validation des Dossiers

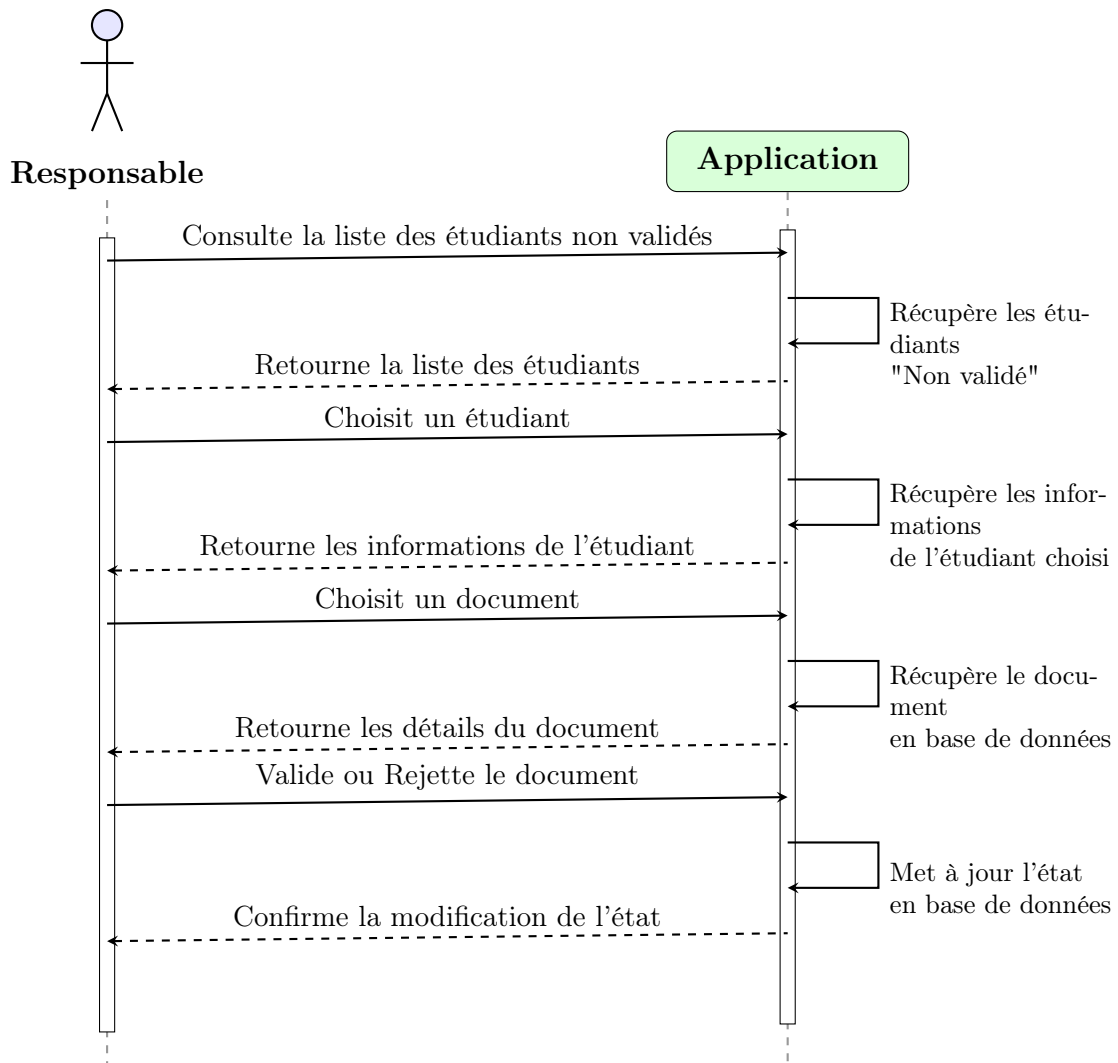
6.1.1 Scénario A1 : Dépôt de Documents par l'Étudiant

Ce scénario décrit le flux d'authentification et de téléchargement sécurisé des pièces justificatives de l'étudiant.



6.1.2 Scénario A2 : Traitement et Validation de Dossier par le Responsable

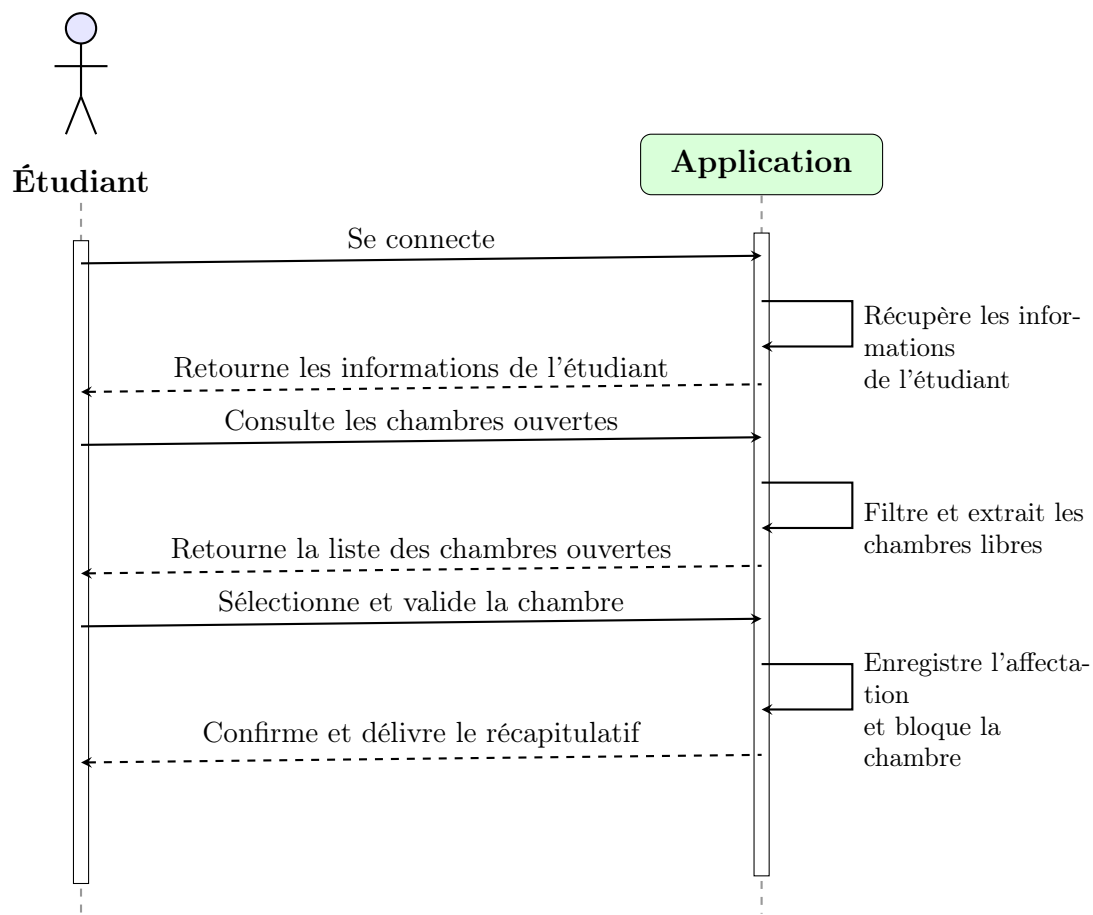
Ce diagramme détaille le flux d'analyse et de validation des pièces justificatives par le responsable d'internat.



6.2 Module B : Réservation et Attribution des Chambres

6.2.1 Scénario B1 : Réservation de Chambre par l'Étudiant

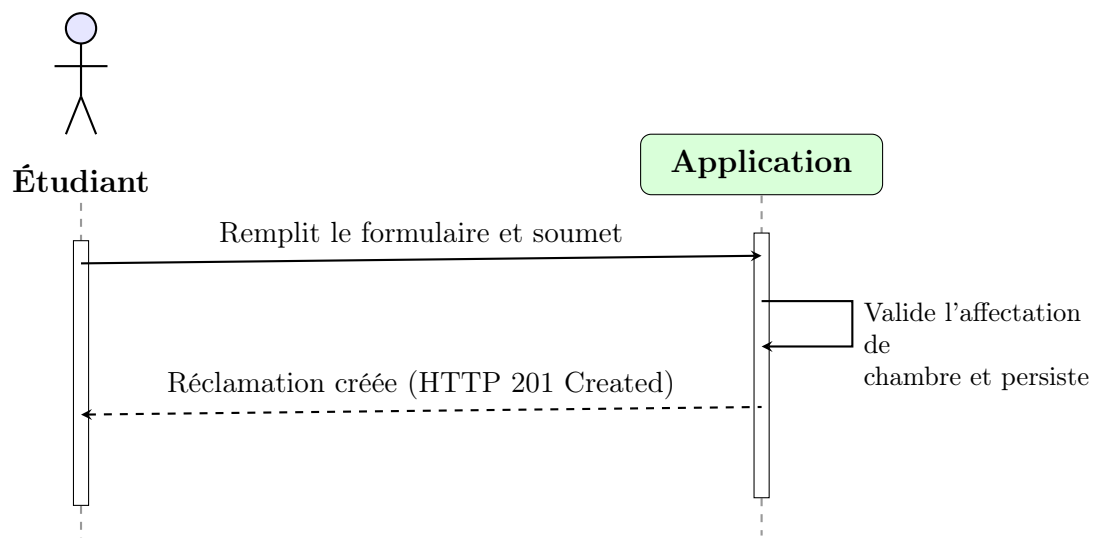
Ce flux permet à l'étudiant de consulter de manière transparente l'inventaire en temps réel et de réserver son affectation.



6.3 Module C : Réclamations et Demandes de Changement

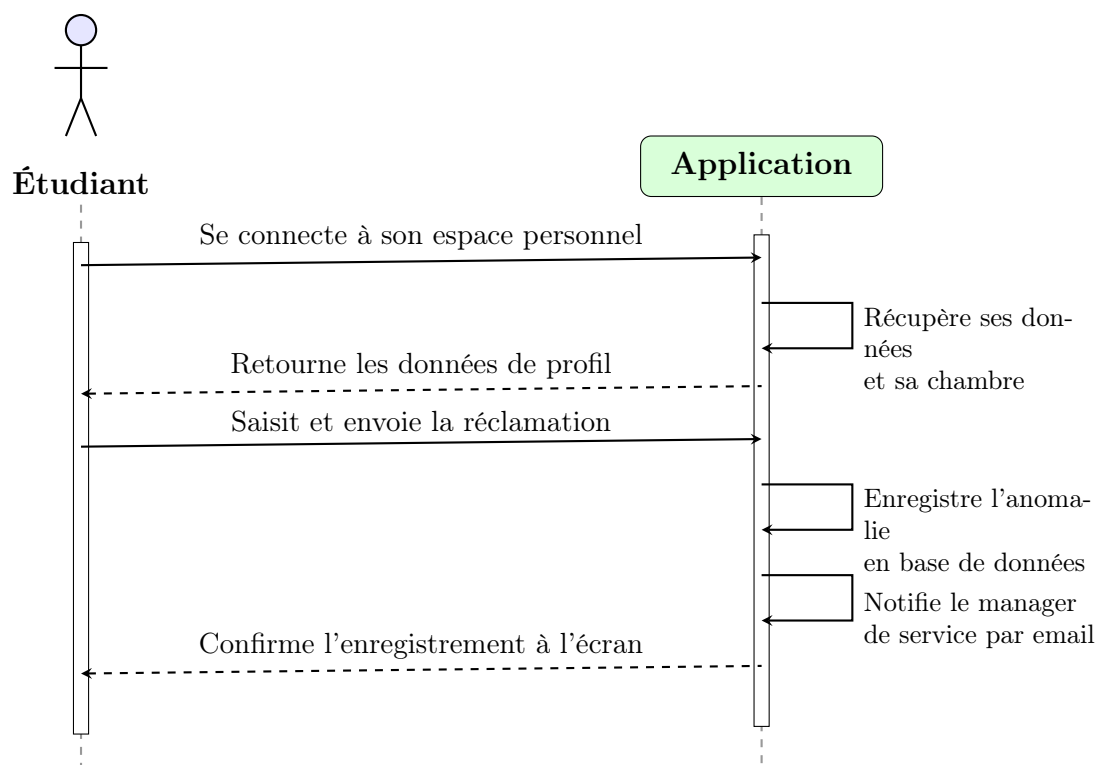
6.3.1 Scénario C1 : Dépôt Direct d'une Réclamation par l'Étudiant

Ce schéma simplifie à l'extrême le parcours utilisateur de signalement d'un dysfonctionnement matériel de terrain.



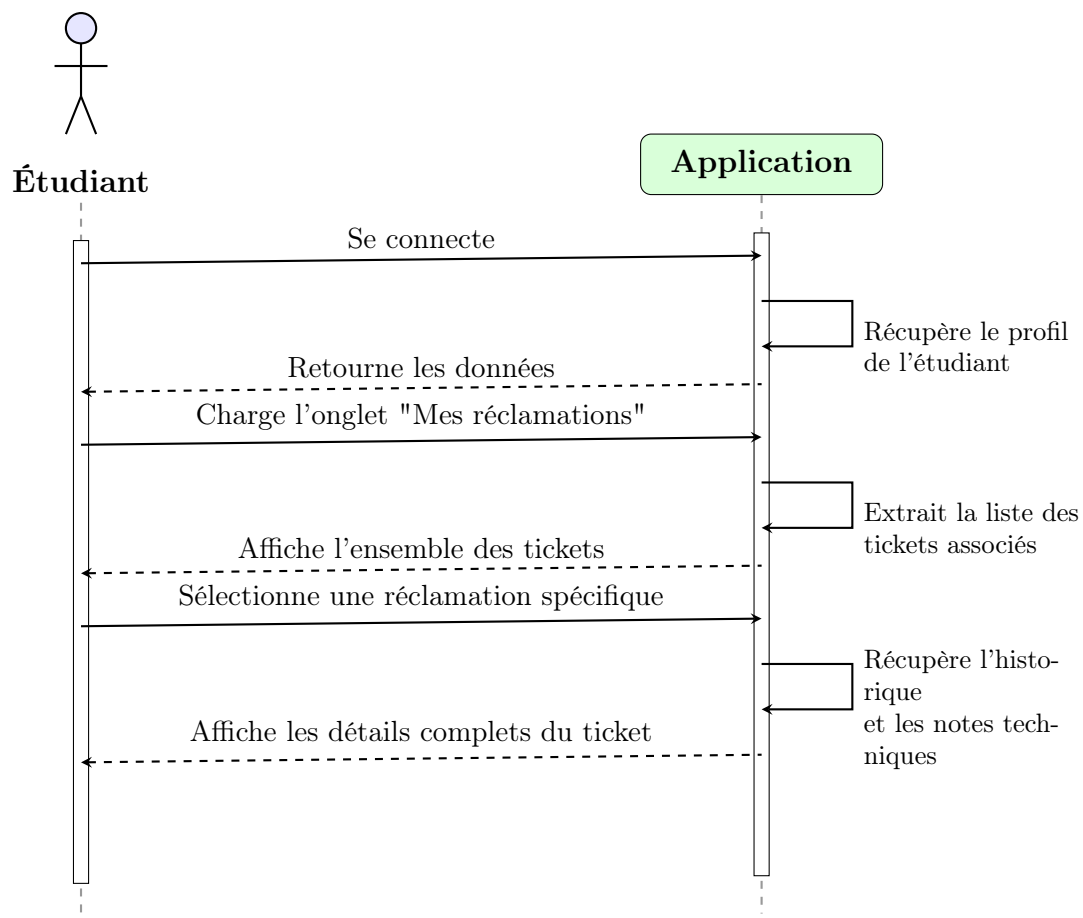
6.3.2 Scénario C2 : Création de Réclamation depuis l'Espace Étudiant

Ce scénario montre le flux de déclaration standardisé au sein du tableau de bord de l'application.



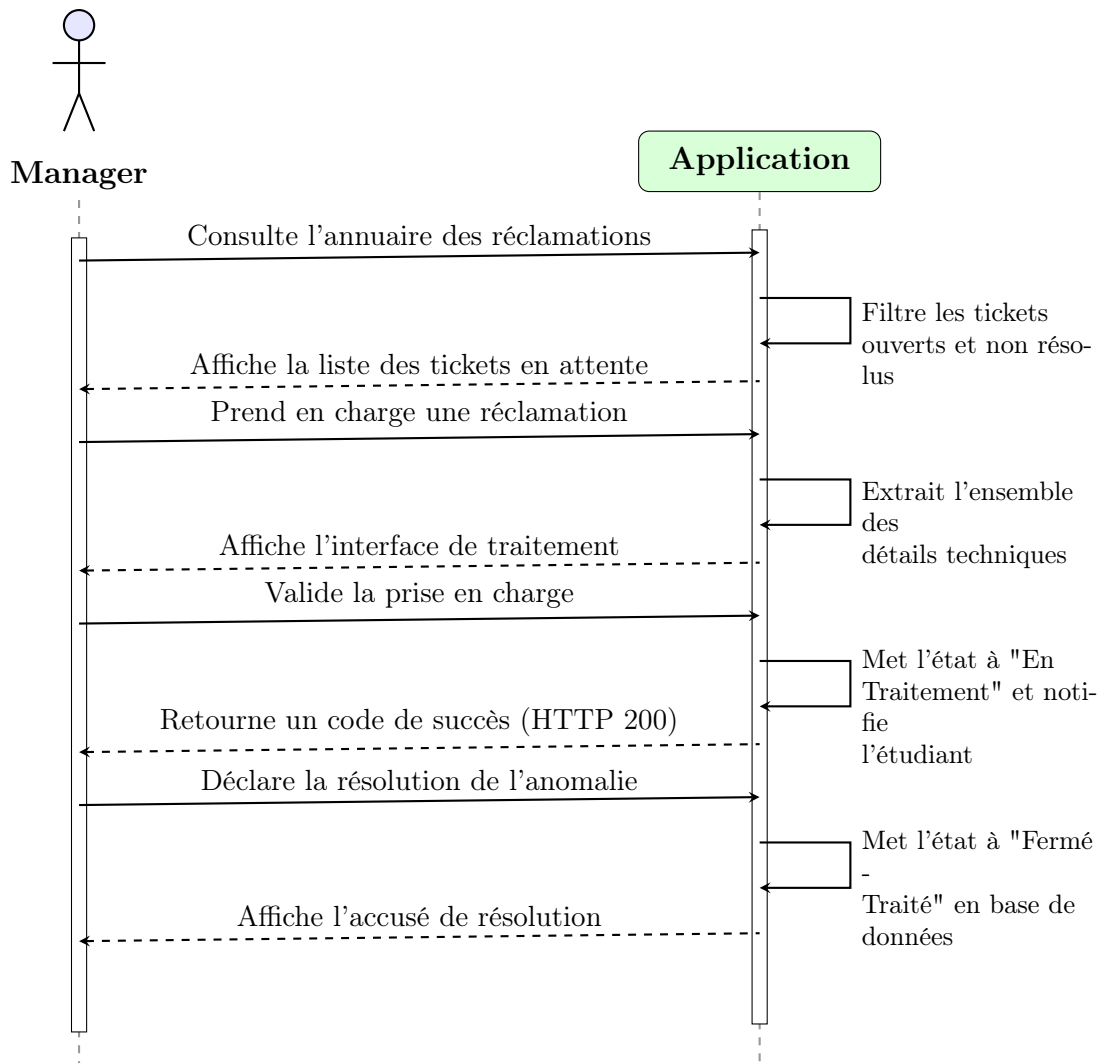
6.3.3 Scénario C3 : Suivi de l'Avancement par l'Étudiant

Visualisation en temps réel de l'état des tickets de maintenance ou d'assistance en cours de traitement.



6.3.4 Scénario C4 : Traitement et Clôture par le Manager de Résidence

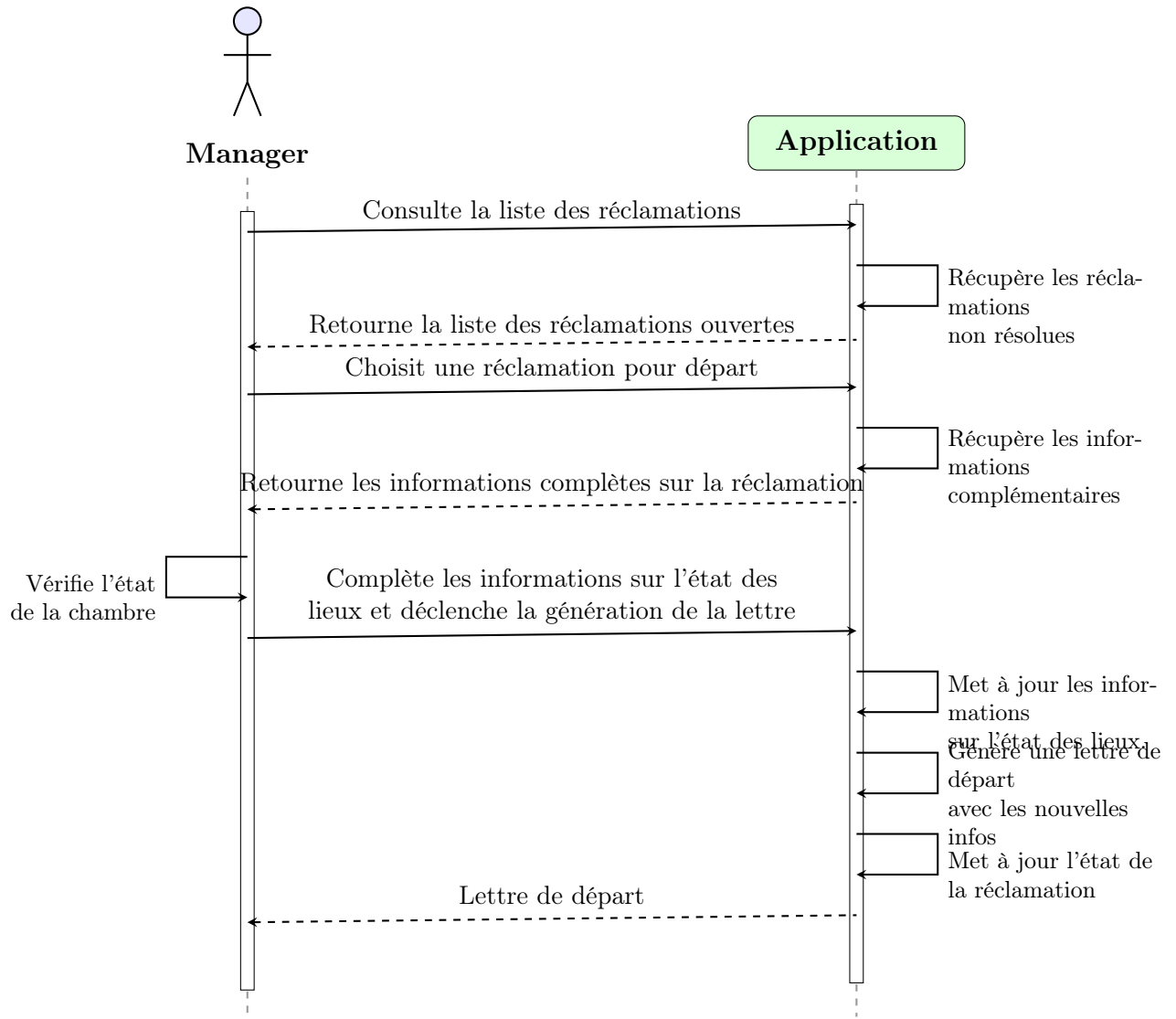
Flux opérationnel de gestion, assignation, mise à jour et résolution des réclamations par le personnel.



6.4 Module D : Gestion des Départs et États des Lieux

6.4.1 Scénario D : Processus Administratif de Libération de Chambre

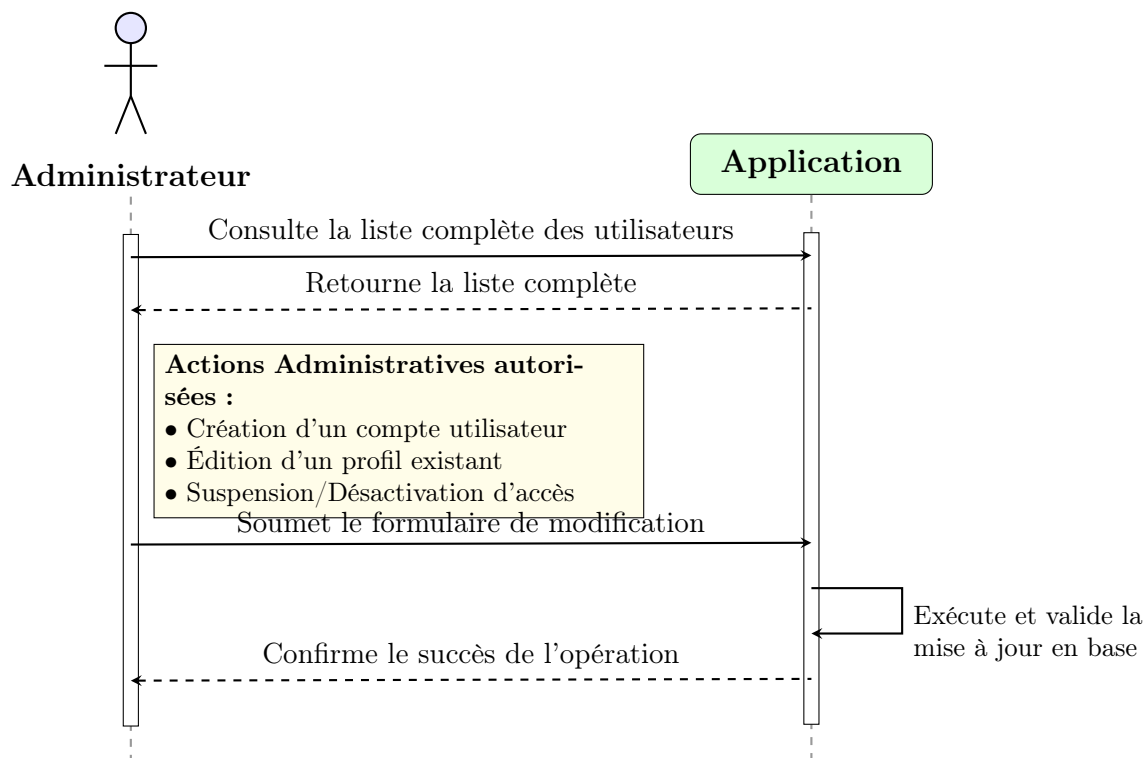
Ce diagramme de séquence décrit la déclaration de sortie d'un résident et la validation d'état de la chambre.



6.5 Module E : Administration, RBAC et Audit

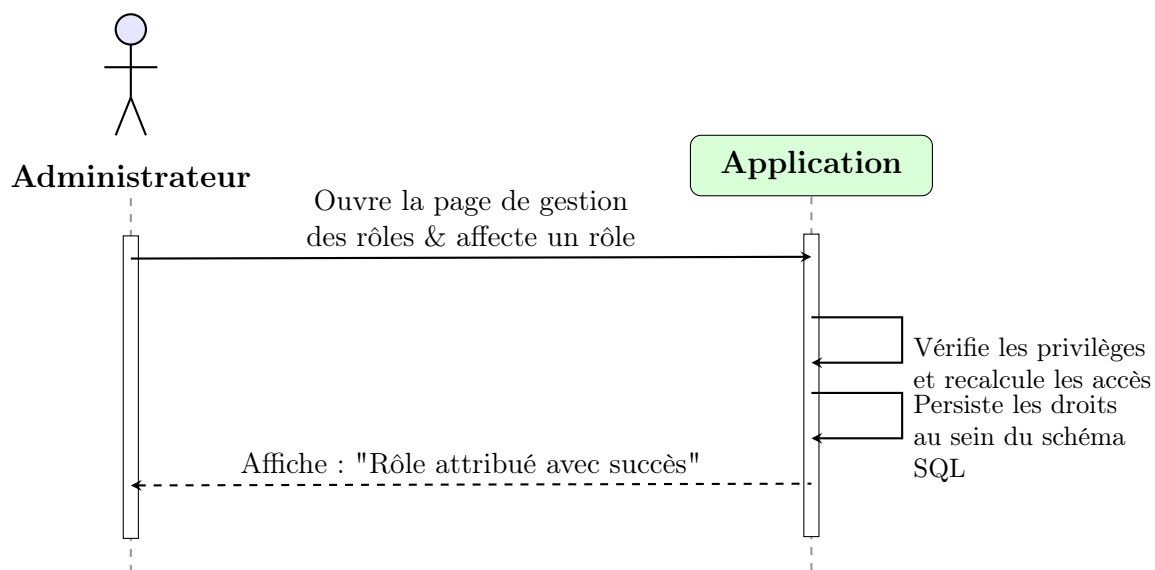
6.5.1 Scénario E1 : Gestion Applicative des Profils Utilisateurs

Contrôle complet des habilitations des comptes et annuaires par l'administrateur du système.



6.5.2 Scénario E2 : Gestion Fine des Rôles (RBAC)

Ce flux standardisé permet l'assignation et la réorganisation des rôles pour sécuriser les données et processus métiers.



6.5.3 Scénario E3 : Consultation des Journaux d'Audit Sécurisés

Ce diagramme modélise l'extraction sécurisée des logs techniques de traçabilité des opérations.

